

Model performance measurement

Confusion Matrix

```
from sklearn.metrics import confusion_matrix
```

```
tn, fp, fn, tp = confusion_matrix([0, 1, 0, 1], [1, 1, 1, 0]).ravel()
(tn, fp, fn, tp)
```

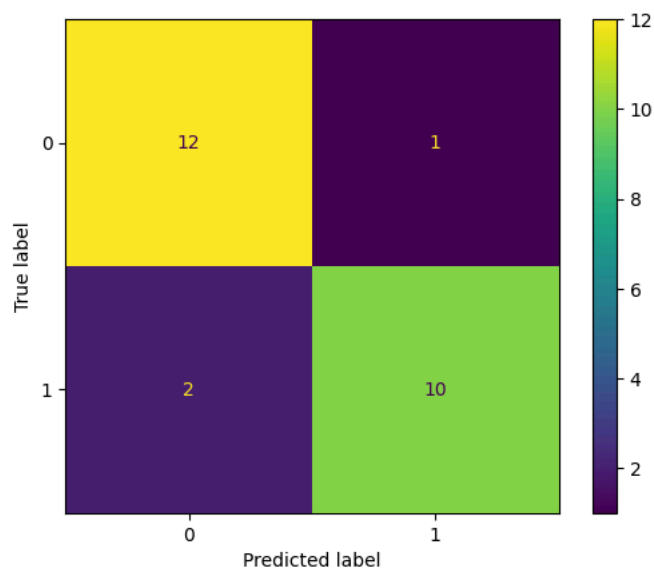
```
(np.int64(0), np.int64(2), np.int64(1), np.int64(1))
```

```
import matplotlib.pyplot as plt
from sklearn.datasets import make_classification
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
X, y = make_classification(random_state=0)
X_train, X_test, y_train, y_test = train_test_split(X, y,
                                                    random_state=0)

clf = SVC(random_state=0)
clf.fit(X_train, y_train)
SVC(random_state=0)
predictions = clf.predict(X_test)
cm = confusion_matrix(y_test, predictions, labels=clf.classes_)
disp = ConfusionMatrixDisplay(confusion_matrix=cm,
                              display_labels=clf.classes_)

disp.plot()

plt.show()
```



F1, Accuracy, Recall, AUC and Precision scores

```
from sklearn.metrics import f1_score

y_true = [0, 1, 2, 0, 1, 2]
y_pred = [0, 2, 1, 0, 0, 1]

print(f'Macro f1 score: {f1_score(y_true, y_pred, average='macro')}')

print(f'Micro F1: {f1_score(y_true, y_pred, average='micro')}')

print(f'Weighted Average F1: {f1_score(y_true, y_pred, average='weighted')}')

print(f'F1 No Average: {f1_score(y_true, y_pred, average=None)}')

y_true = [0, 0, 0, 0, 0, 0]
y_pred = [0, 0, 0, 0, 0, 0]
f1_score(y_true, y_pred, zero_division=1)

# multilabel classification
```

```
y_true = [[0, 0, 0], [1, 1, 1], [0, 1, 1]]
y_pred = [[0, 0, 0], [1, 1, 1], [1, 1, 0]]
print(f"F1 No Average: {f1_score(y_true, y_pred, average=None)}")
```

```
Macro f1 score: 0.26666666666666666
Micro F1: 0.3333333333333333
Weighted Average F1: 0.26666666666666666
F1 No Average: [0.8 0. 0. ]
F1 No Average: [0.66666667 1. 0.66666667]
```

```
from sklearn.metrics import accuracy_score
y_pred = [0, 2, 1, 3]
y_true = [0, 1, 2, 3]
accuracy_score(y_true, y_pred)
```

0.5

```
from sklearn.metrics import precision_score
y_true = [0, 1, 2, 0, 1, 2]
y_pred = [0, 2, 1, 0, 0, 1]
precision_score(y_true, y_pred, average='macro')
```

0.2222222222222222

```
from sklearn.metrics import recall_score
y_true = [0, 1, 2, 0, 1, 2]
y_pred = [0, 2, 1, 0, 0, 1]
recall_score(y_true, y_pred, average='macro')
```

0.3333333333333333

```
from sklearn.metrics import classification_report
y_true = [0, 1, 2, 2, 2]
y_pred = [0, 0, 2, 2, 1]
target_names = ['class 0', 'class 1', 'class 2']
print(classification_report(y_true, y_pred, target_names=target_names))
```

	precision	recall	f1-score	support
class 0	0.50	1.00	0.67	1
class 1	0.00	0.00	0.00	1
class 2	1.00	0.67	0.80	3
accuracy			0.60	5
macro avg	0.50	0.56	0.49	5
weighted avg	0.70	0.60	0.61	5

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score
X, y = load_breast_cancer(return_X_y=True)
clf = LogisticRegression(solver="liblinear", random_state=0).fit(X, y)
roc_auc_score(y, clf.predict_proba(X)[:, 1])
```

np.float64(0.994767718408118)

```
#multiclass case
from sklearn.datasets import load_iris
X, y = load_iris(return_X_y=True)
clf = LogisticRegression(solver="liblinear").fit(X, y)
roc_auc_score(y, clf.predict_proba(X), multi_class='ovr')
```

np.float64(0.9913333333333334)

▼ AUC Parameter Changes

```
from sklearn.datasets import load_breast_cancer
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import roc_auc_score

X, y = load_breast_cancer(return_X_y=True)

clf = LogisticRegression(
    solver="lbfgs",
    C=10,
    random_state=42,
```

```

        max_iter=1000
    )

    clf.fit(X, y)

    print("AUC =", roc_auc_score(y, clf.predict_proba(X)[: , 1]))

```

AUC = 0.9953358701971354
 /usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge. Increase the number of iterations.
 STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```
from sklearn.linear_model import LogisticRegression
```

```

clf = LogisticRegression(
    solver="lbfgs",
    C=10,
    random_state=42,
    max_iter=5000
)

```

```
clf.fit(X, y)
```

```

auc = roc_auc_score(y, clf.predict_proba(X)[: , 1])
print("AUC =", auc)

```

AUC = 0.9959568733153639
 /usr/local/lib/python3.12/dist-packages/sklearn/linear_model/_logistic.py:465: ConvergenceWarning: lbfgs failed to converge. Increase the number of iterations.
 STOP: TOTAL NO. OF ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:

<https://scikit-learn.org/stable/modules/preprocessing.html>

Please also refer to the documentation for alternative solver options:

https://scikit-learn.org/stable/modules/linear_model.html#logistic-regression

```
n_iter_i = _check_optimize_result(
```

```

from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

```

```

clf = make_pipeline(
    StandardScaler(),
    LogisticRegression(
        solver="lbfgs",
        C=10,
        random_state=42,
        max_iter=5000
    )
)

```

```
clf.fit(X, y)
```

```

auc = roc_auc_score(y, clf.predict_proba(X)[: , 1])
print("AUC =", auc)

```

AUC = 0.9986126526082131

```

solver="liblinear"
random_state=0

```

```

clf = LogisticRegression(
    solver="lbfgs",
    random_state=0,
    max_iter=5000
).fit(X, y)

```

```
roc_auc_score(y, clf.predict_proba(X)[: , 1])
```

```
np.float64(0.9946752285819989)
```

```

clf = LogisticRegression(
    solver="liblinear",
    C=0.1,
    random_state=0
).fit(X, y)

```

```
roc_auc_score(y, clf.predict_proba(X)[: ,1])

np.float64(0.9914116590032238)
```

▼ AUC Experiment Findings

The baseline model achieved an AUC score of 0.9953, which indicates excellent classification performance. When the solver was changed from liblinear to lbfgs, this had almost no effect on the model's ability to distinguish between classes. Reducing the regularisation parameter ($C=0.1$) slightly decreased the AUC as the model became more constrained. Increasing C to 10 slightly improved the AUC compared with $C=0.1$, producing performance very close to the original model.

```
import numpy as np
import matplotlib.pyplot as plt
from itertools import cycle

from sklearn import svm, datasets
from sklearn.metrics import roc_curve, auc
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import label_binarize
from sklearn.multiclass import OneVsRestClassifier
from sklearn.metrics import roc_auc_score

# Import some data to play with
iris = datasets.load_iris()
X = iris.data
y = iris.target

# Binarize the output
y = label_binarize(y, classes=[0, 1, 2])
n_classes = y.shape[1]

# Add noisy features to make the problem harder
random_state = np.random.RandomState(0)
n_samples, n_features = X.shape
X = np.c_[X, random_state.randn(n_samples, 200 * n_features)]

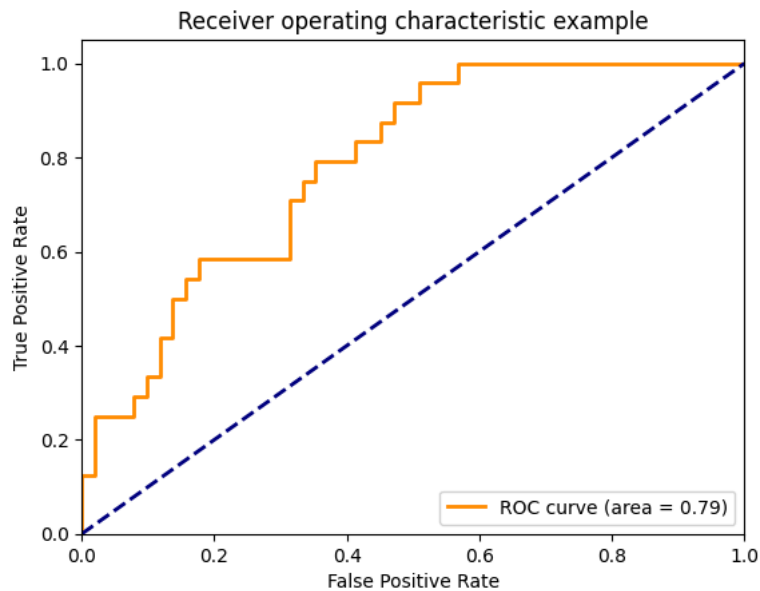
# shuffle and split training and test sets
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.5, random_state=0)

# Learn to predict each class against the other
classifier = OneVsRestClassifier(
    svm.SVC(kernel="linear", probability=True, random_state=random_state)
)
y_score = classifier.fit(X_train, y_train).decision_function(X_test)

# Compute ROC curve and ROC area for each class
fpr = dict()
tpr = dict()
roc_auc = dict()
for i in range(n_classes):
    fpr[i], tpr[i], _ = roc_curve(y_test[:, i], y_score[:, i])
    roc_auc[i] = auc(fpr[i], tpr[i])

# Compute micro-average ROC curve and ROC area
fpr["micro"], tpr["micro"], _ = roc_curve(y_test.ravel(), y_score.ravel())
roc_auc["micro"] = auc(fpr["micro"], tpr["micro"])
```

```
plt.figure()
lw = 2
plt.plot(
    fpr[2],
    tpr[2],
    color="darkorange",
    lw=lw,
    label="ROC curve (area = %0.2f)" % roc_auc[2],
)
plt.plot([0, 1], [0, 1], color="navy", lw=lw, linestyle="--")
plt.xlim([0.0, 1.0])
plt.ylim([0.0, 1.05])
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Receiver operating characteristic example")
plt.legend(loc="lower right")
plt.show()
```



```
from sklearn.metrics import log_loss
log_loss(["spam", "ham", "ham", "spam"], [[.1, .9], [.9, .1], [.8, .2], [.35, .65]])
```

```
0.21616187468057912
```

Regression metrics

RMSE

```
from sklearn.metrics import mean_squared_error
y_true = [3, -0.5, 2, 7]
y_pred = [2.5, 0.0, 2, 8]
mean_squared_error(y_true, y_pred)
```

```
0.375
```

MAE

```
from sklearn.metrics import mean_absolute_error
y_true = [3, -0.5, 2, 7]
y_pred = [2.5, 0.0, 2, 8]
mean_absolute_error(y_true, y_pred)
```

```
0.5
```

r squared

```
from sklearn.metrics import r2_score
r2_score(y_true, y_pred)
```

```
0.375
```

R² Experiment

```
print(y_true)
print(y_pred)

print(len(y_true))
print(len(y_pred))

[0, 1, 2, 2, 2]
[2.9, -0.4, 2.1, 6.9]
5
4
```

```
from sklearn.metrics import r2_score
```

```
y_true = [3, -0.5, 2, 7]
y_pred = [2.5, 0.0, 2, 8]

print(r2_score(y_true, y_pred))
```

0.9486081370449679

```
from sklearn.metrics import r2_score

y_true = [3, -0.5, 2, 7]
y_pred = [1, 3, -2, 12]

print(r2_score(y_true, y_pred))
```

-0.9614561027837258

Start coding or [generate](#) with AI.

✓ R² Experiment Findings

The original R² score (0.375) indicated moderate predictive performance. When the predicted values were improved, the R² score increased to 0.9486, demonstrating that the model explained almost all of the variation in the target values. When using poor predictions, the R² decreased to -0.9615, demonstrating that inaccurate predictions can make the model perform worse than a simple baseline prediction.

Start coding or [generate](#) with AI.

✓ Conclusion

The experiments conducted demonstrated how changes to the model parameters and prediction accuracy affect performance metrics.

In terms of the Logistic Regression classifier, all parameter settings produced very high AUC values (above 0.99), demonstrating that the model classified the breast cancer dataset extremely well. Changing the solver had very little impact, while adjusting the regularisation parameter (C) produced small differences in performance. A lower C value slightly reduced the AUC due to stronger regularisation, whereas increasing C restored performance close to the baseline.

In terms of the regression, the R² score was highly dependent on prediction accuracy. Predictions that closely matched the true values increased the R² score from 0.375 to 0.9486, which indicates an excellent fit. However, poor predictions produced a negative R² score (-0.9615), demonstrating that the model performed worse than simply predicting the average target value.

In conclusion, these experiments demonstrate that selecting appropriate model parameters and producing accurate predictions are essential for achieving strong model performance.

Start coding or [generate](#) with AI.